

DESIGN AND ANALYSIS OF ALGORITHMS

DAY 7 – GREEDY ALGORITHMS

Name: N. PARNITA

Register No.: 192525322

Course code: CSA0617

Course Name: Design and Analysis of Algorithm

TABLE OF CONTENTS

Experiment 1: Coin Change – Supermarket Billing

Experiment 2: Coin Change – Parking Meter

Experiment 3: Coin Change – Ticket Counter

Experiment 4: Fractional Knapsack – Food Delivery Truck

Experiment 5: Fractional Knapsack – Gold Transport

Experiment 6: Fractional Knapsack – Relief Material Loading

Experiment 7: Job Sequencing – Software Company Tasks

Experiment 8: Job Sequencing – Exam Paper Evaluation

Experiment 9: Job Sequencing – Printing Press Orders

Experiment 10: Job Sequencing – Freelance Projects

Experiment 11: Huffman Coding – Text Compression

Experiment 12: Huffman Coding – Message Encoding

Experiment 13: Huffman Coding – Sensor Data Compression

Experiment 14: Coin Change – Vending Machine

Experiment 15: Fractional Knapsack – Warehouse Packing

Experiment 1: Coin Change – Supermarket Billing

Objective: To return the minimum number of coins for a customer's balance amount using the available denominations.

Problem Statement: To return the minimum number of coins for a customer's balance amount using the available denominations.

Sample Input: Coins = [1, 2, 5, 10], Amount = 28

Sample Output: 10 10 5 2 1; Minimum coins = 5

Time Complexity: $O(n \log n + A/c)$ time for greedy selection after sorting.

Explanation: The greedy approach sorts denominations in descending order and repeatedly selects the largest denomination that does not exceed the remaining amount.

Python Program

```
def coin_change(coins, amount):
    coins = sorted(coins, reverse=True)
    result = []

    for coin in coins:
        while amount >= coin:
            amount -= coin
            result.append(coin)

    return result

coins = list(map(int, input("Enter coins: ").split()))
amount = int(input("Enter amount: "))

result = coin_change(coins, amount)
print("Coins used:", result)
print("Minimum coins =", len(result))
```

Execution / Output

```
Enter coins: 1 2 5 10
Enter amount: 28
Coins used: [10, 10, 5, 2, 1]
Minimum coins = 5
```

Result: The program was implemented and the expected result was obtained.

Experiment 2: Coin Change – Parking Meter

Objective: To find the minimum coins required for a parking meter using fixed denominations.

Problem Statement: To find the minimum coins required for a parking meter using fixed denominations.

Sample Input: Coins = [1, 5, 10, 25], Amount = 63

Sample Output: 25 25 10 1 1 1; Minimum coins = 6

Time Complexity: $O(n \log n)$ time after sorting.

Explanation: The greedy approach sorts denominations in descending order and repeatedly selects the largest denomination that does not exceed the remaining amount.

Python Program

```
def coin_change(coins, amount):
    coins = sorted(coins, reverse=True)
    result = []

    for coin in coins:
        count = amount // coin
        result.extend([coin] * count)
        amount %= coin

    return result

coins = list(map(int, input("Enter coins: ").split()))
amount = int(input("Enter amount: "))

result = coin_change(coins, amount)
print("Coins used:", result)
print("Minimum coins =", len(result))
```

Execution / Output

```
Enter coins: 1 5 10 25
Enter amount: 63
Coins used: [25, 25, 10, 1, 1, 1]
Minimum coins = 6
```

Result: The program was implemented and the expected result was obtained.

Experiment 3: Coin Change – Ticket Counter

Objective: To return change using available denominations while minimizing the number of coins.

Problem Statement: To return change using available denominations while minimizing the number of coins.

Sample Input: Coins = [1, 3, 7, 10], Amount = 24

Sample Output: 10 10 3 1; Minimum coins = 4

Time Complexity: $O(n \log n + A/c)$ time.

Explanation: The greedy approach sorts denominations in descending order and repeatedly selects the largest denomination that does not exceed the remaining amount.

Python Program

```
def coin_change(coins, amount):
    coins = sorted(coins, reverse=True)
    result = []

    for coin in coins:
        while amount >= coin:
            amount -= coin
            result.append(coin)

    return result

coins = list(map(int, input("Enter coins: ").split()))
amount = int(input("Enter amount: "))

result = coin_change(coins, amount)
print("Coins used:", result)
print("Minimum coins =", len(result))
```

Execution / Output

```
Enter coins: 1 3 7 10
Enter amount: 24
Coins used: [10, 10, 3, 1]
Minimum coins = 4
```

Result: The program was implemented and the expected result was obtained.

Experiment 4: Fractional Knapsack – Food Delivery Truck

Objective: To maximize profit by selecting whole or fractional parts of food packets within a weight capacity.

Problem Statement: To maximize profit by selecting whole or fractional parts of food packets within a weight capacity.

Sample Input: Weights = [10, 20, 30], Profits = [60, 100, 120], Capacity = 50

Sample Output: Maximum profit = 240

Time Complexity: $O(n \log n)$ time and $O(n)$ space.

Explanation: Items are sorted by profit-to-weight ratio. The highest-ratio items are selected first, and a fraction of the next item may be taken when the remaining capacity is smaller than its weight.

Python Program

```
def fractional_knapsack(weights, profits, capacity):
    items = []

    for w, p in zip(weights, profits):
        items.append((p / w, w, p))

    items.sort(reverse=True)
    total = 0

    for ratio, weight, profit in items:
        if capacity == 0:
            break

        take = min(weight, capacity)
        total += ratio * take
        capacity -= take

    return total

weights = list(map(int, input("Enter weights: ").split()))
profits = list(map(int, input("Enter profits: ").split()))
capacity = int(input("Enter capacity: "))

print("Maximum profit =", fractional_knapsack(weights, profits, capacity))
```

Execution / Output

```
Enter weights: 10 20 30
Enter profits: 60 100 120
Enter capacity: 50
Maximum profit = 240.0
```

Result: The program was implemented and the expected result was obtained.

Experiment 5: Fractional Knapsack – Gold Transport

Objective: To maximize the value of gold transported under a maximum weight limit.

Problem Statement: To maximize the value of gold transported under a maximum weight limit.

Sample Input: Weights = [5, 10, 15], Profits = [30, 40, 45], Capacity = 20

Sample Output: Maximum profit = 80

Time Complexity: $O(n \log n)$ time and $O(n)$ space.

Explanation: Items are sorted by profit-to-weight ratio. The highest-ratio items are selected first, and a fraction of the next item may be taken when the remaining capacity is smaller than its weight.

Python Program

```
def fractional_knapsack(weights, profits, capacity):
    items = sorted(
        [(p / w, w, p) for w, p in zip(weights, profits)],
        reverse=True
    )

    total = 0

    for ratio, weight, profit in items:
        take = min(weight, capacity)
        total += ratio * take
        capacity -= take

        if capacity == 0:
            break

    return total

weights = list(map(int, input("Enter weights: ").split()))
profits = list(map(int, input("Enter profits: ").split()))
capacity = int(input("Enter capacity: "))

print("Maximum profit =", fractional_knapsack(weights, profits, capacity))
```

Execution / Output

```
Enter weights: 5 10 15
Enter profits: 30 40 45
Enter capacity: 20
Maximum profit = 80.0
```

Result: The program was implemented and the expected result was obtained.

Experiment 6: Fractional Knapsack – Relief Material Loading

Objective: To maximize the importance value of relief materials loaded into a limited-capacity vehicle.

Problem Statement: To maximize the importance value of relief materials loaded into a limited-capacity vehicle.

Sample Input: Weights = [4, 8, 2, 6], Profits = [20, 40, 14, 30], Capacity = 10

Sample Output: Maximum profit = 54

Time Complexity: $O(n \log n)$ time and $O(n)$ space.

Explanation: Items are sorted by profit-to-weight ratio. The highest-ratio items are selected first, and a fraction of the next item may be taken when the remaining capacity is smaller than its weight.

Python Program

```
def fractional_knapsack(weights, profits, capacity):
    items = sorted(
        [(p / w, w, p) for w, p in zip(weights, profits)],
        reverse=True
    )

    total = 0

    for ratio, weight, profit in items:
        take = min(weight, capacity)
        total += ratio * take
        capacity -= take

        if capacity == 0:
            break

    return total

weights = list(map(int, input("Enter weights: ").split()))
profits = list(map(int, input("Enter profits: ").split()))
capacity = int(input("Enter capacity: "))

print("Maximum profit =", fractional_knapsack(weights, profits, capacity))
```

Execution / Output

```
Enter weights: 4 8 2 6
Enter profits: 20 40 14 30
Enter capacity: 10
Maximum profit = 54.0
```

Result: The program was implemented and the expected result was obtained.

Experiment 7: Job Sequencing – Software Company Tasks

Objective: To schedule jobs before their deadlines to maximize total profit, with at most one job completed per time slot.

Problem Statement: To schedule jobs before their deadlines to maximize total profit, with at most one job completed per time slot.

Sample Input: Jobs = A B C D; Deadlines = [2,1,2,1]; Profits = [100,19,27,25]

Sample Output: Selected jobs = C A; Maximum profit = 127

Time Complexity: $O(n \log n + nD)$ time, where D is the maximum deadline.

Explanation: Jobs are sorted by profit in descending order. Each job is placed in the latest available slot before its deadline to preserve earlier slots for other jobs.

Python Program

```
def job_sequencing(jobs, deadlines, profits):
    data = sorted(
        zip(jobs, deadlines, profits),
        key=lambda x: x[2],
        reverse=True
    )
    max_deadline = max(deadlines)
    slots = [None] * (max_deadline + 1)
    total = 0
    selected = []

    for job, deadline, profit in data:
        for t in range(min(deadline, max_deadline), 0, -1):
            if slots[t] is None:
                slots[t] = job
                selected.append(job)
                total += profit
                break

    return selected, total

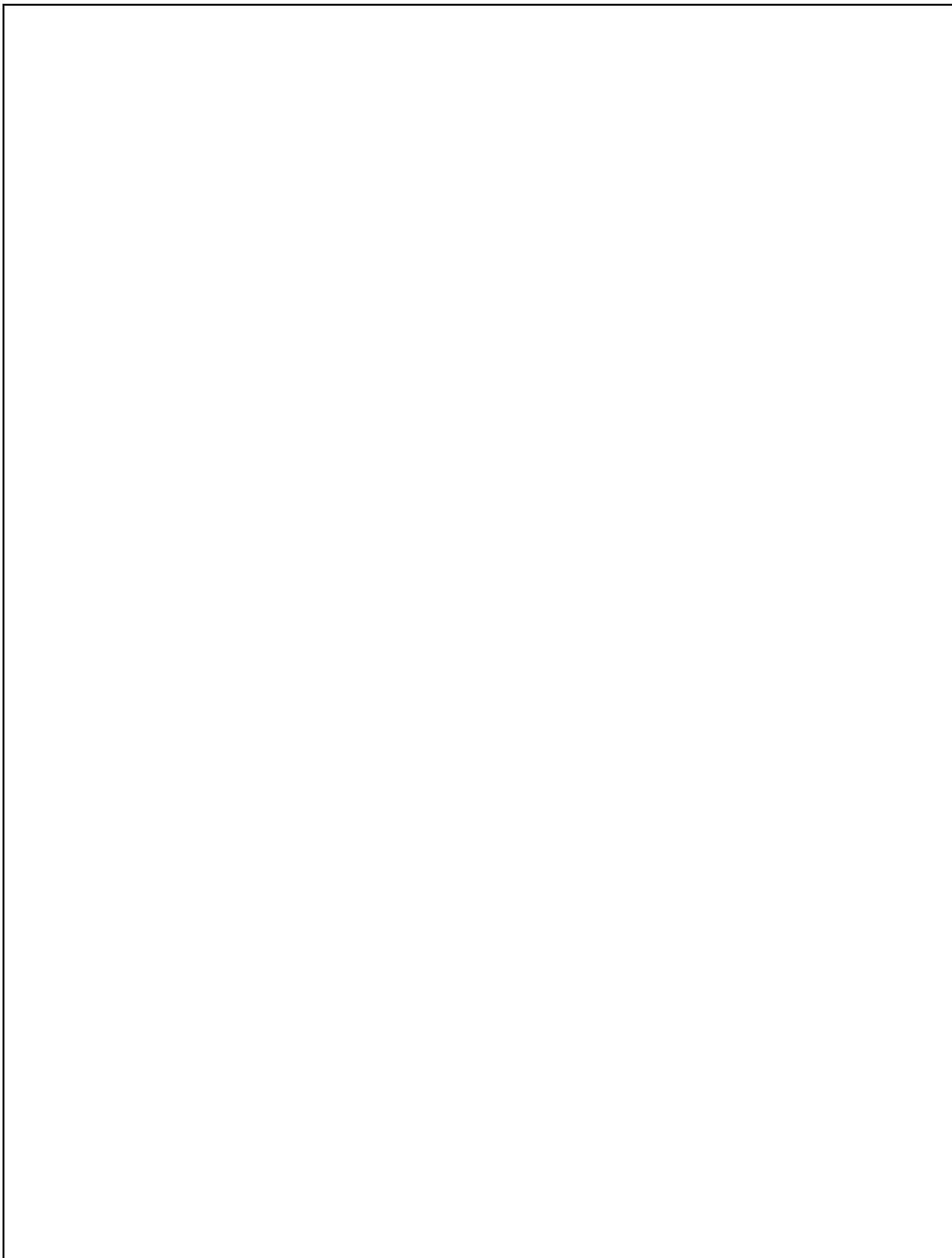
jobs = input("Enter jobs: ").split()
deadlines = list(map(int, input("Enter deadlines: ").split()))
profits = list(map(int, input("Enter profits: ").split()))

selected, total = job_sequencing(jobs, deadlines, profits)
print("Selected jobs =", selected)
print("Maximum profit =", total)
```

Execution / Output

```
Enter jobs: A B C D
Enter deadlines: 2 1 2 1
Enter profits: 100 19 27 25
Selected jobs = ['A', 'C']
Maximum profit = 127
```

Result: The program was implemented and the expected result was obtained.



Experiment 8: Job Sequencing – Exam Paper Evaluation

Objective: To schedule evaluation jobs before deadlines to maximize reward.

Problem Statement: To schedule evaluation jobs before deadlines to maximize reward.

Sample Input: Jobs = J1 J2 J3 J4 J5; Deadlines = [2,1,2,1,3]; Profits = [20,15,10,5,1]

Sample Output: Selected jobs = J2 J1 J5; Maximum profit = 36

Time Complexity: $O(n \log n + nD)$ time.

Explanation: Jobs are sorted by profit in descending order. Each job is placed in the latest available slot before its deadline to preserve earlier slots for other jobs.

Python Program

```
def job_sequencing(jobs, deadlines, profits):
    data = sorted(zip(jobs, deadlines, profits),
                  key=lambda x: x[2], reverse=True)

    slots = [None] * (max(deadlines) + 1)
    total = 0
    selected = []

    for job, deadline, profit in data:
        for t in range(deadline, 0, -1):
            if slots[t] is None:
                slots[t] = job
                selected.append(job)
                total += profit
                break

    return selected, total

jobs = input("Enter jobs: ").split()
deadlines = list(map(int, input("Enter deadlines: ").split()))
profits = list(map(int, input("Enter profits: ").split()))

selected, total = job_sequencing(jobs, deadlines, profits)
print("Selected jobs =", selected)
print("Maximum profit =", total)
```

Execution / Output

```
Enter jobs: J1 J2 J3 J4 J5
Enter deadlines: 2 1 2 1 3
Enter profits: 20 15 10 5 1
Selected jobs = ['J1', 'J2', 'J5']
Maximum profit = 36
```

Result: The program was implemented and the expected result was obtained.

Experiment 9: Job Sequencing – Printing Press Orders

Objective: To schedule printing orders based on deadlines and profit.

Problem Statement: To schedule printing orders based on deadlines and profit.

Sample Input: Jobs = A B C D E; Deadlines = [2,1,2,1,3]; Profits = [100,50,10,20,30]

Sample Output: Selected jobs = B A E; Maximum profit = 180

Time Complexity: $O(n \log n + nD)$ time.

Explanation: Jobs are sorted by profit in descending order. Each job is placed in the latest available slot before its deadline to preserve earlier slots for other jobs.

Python Program

```
def job_sequencing(jobs, deadlines, profits):
    data = sorted(zip(jobs, deadlines, profits),
                  key=lambda x: x[2], reverse=True)

    slots = [None] * (max(deadlines) + 1)
    total = 0
    selected = []

    for job, deadline, profit in data:
        for t in range(deadline, 0, -1):
            if slots[t] is None:
                slots[t] = job
                selected.append(job)
                total += profit
                break

    return selected, total

jobs = input("Enter jobs: ").split()
deadlines = list(map(int, input("Enter deadlines: ").split()))
profits = list(map(int, input("Enter profits: ").split()))

selected, total = job_sequencing(jobs, deadlines, profits)
print("Selected jobs =", selected)
print("Maximum profit =", total)
```

Execution / Output

```
Enter jobs: A B C D E
Enter deadlines: 2 1 2 1 3
Enter profits: 100 50 10 20 30
Selected jobs = ['A', 'B', 'E']
Maximum profit = 180
```

Result: The program was implemented and the expected result was obtained.

Experiment 10: Job Sequencing – Freelance Projects

Objective: To select freelance projects that maximize income when at most one project can be completed per day.

Problem Statement: To select freelance projects that maximize income when at most one project can be completed per day.

Sample Input: Jobs = P1 P2 P3 P4; Deadlines = [1,1,2,2]; Profits = [40,30,20,10]

Sample Output: Selected jobs = P1 P3; Maximum profit = 60

Time Complexity: $O(n \log n + nD)$ time.

Explanation: Jobs are sorted by profit in descending order. Each job is placed in the latest available slot before its deadline to preserve earlier slots for other jobs.

Python Program

```
def job_sequencing(jobs, deadlines, profits):
    data = sorted(zip(jobs, deadlines, profits),
                  key=lambda x: x[2], reverse=True)

    slots = [None] * (max(deadlines) + 1)
    total = 0
    selected = []

    for job, deadline, profit in data:
        for t in range(deadline, 0, -1):
            if slots[t] is None:
                slots[t] = job
                selected.append(job)
                total += profit
                break

    return selected, total

jobs = input("Enter jobs: ").split()
deadlines = list(map(int, input("Enter deadlines: ").split()))
profits = list(map(int, input("Enter profits: ").split()))

selected, total = job_sequencing(jobs, deadlines, profits)
print("Selected jobs =", selected)
print("Maximum profit =", total)
```

Execution / Output

```
Enter jobs: P1 P2 P3 P4
Enter deadlines: 1 1 2 2
Enter profits: 40 30 20 10
Selected jobs = ['P1', 'P3']
Maximum profit = 60
```

Result: The program was implemented and the expected result was obtained.

Experiment 11: Huffman Coding – Text Compression

Objective: To generate Huffman codes for characters according to their frequencies for file compression.

Problem Statement: To generate Huffman codes for characters according to their frequencies for file compression.

Sample Input: Characters = a b c d e f; Frequencies = [5,9,12,13,16,45]

Sample Output: f: 0; c: 100; d: 101; a: 1100; b: 1101; e: 111

Time Complexity: $O(n \log n)$ time and $O(n)$ space.

Explanation: Huffman coding repeatedly combines the two least-frequent nodes to build a prefix-code tree. More frequent symbols receive shorter codes.

Python Program

```
import heapq
def huffman_codes(chars, freq):
    heap = [[f, [ch, ""]] for ch, f in zip(chars, freq)]
    heapq.heapify(heap)

    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)

        for pair in left[1:]:
            pair[1] = "0" + pair[1]
        for pair in right[1:]:
            pair[1] = "1" + pair[1]
        heapq.heappush(
            heap,
            [left[0] + right[0]] + left[1:] + right[1:]
        )
    return dict(sorted(heap[0][1:], key=lambda x: x[0]))
chars = input("Enter characters: ").split()
freq = list(map(int, input("Enter frequencies: ").split()))
codes = huffman_codes(chars, freq)
for ch, code in codes.items():
    print(f"{ch}: {code}")
```

Execution / Output

```
Enter characters: a b c d e f
Enter frequencies: 5 9 12 13 16 45
f: 0
c: 100
d: 101
a: 1100
b: 1101
e: 111
```

Result: The program was implemented and the expected result was obtained.

Experiment 12: Huffman Coding – Message Encoding

Objective: To generate Huffman codes for frequently used symbols in a messaging application.

Problem Statement: To generate Huffman codes for frequently used symbols in a messaging application.

Sample Input: Characters = A B C D; Frequencies = [10,20,30,40]

Sample Output: D: 0; C: 10; A: 110; B: 111

Time Complexity: $O(n \log n)$ time and $O(n)$ space.

Explanation: Huffman coding repeatedly combines the two least-frequent nodes to build a prefix-code tree. More frequent symbols receive shorter codes.

Python Program

```
import heapq

def huffman_codes(chars, freq):
    heap = [[f, [[ch, ""]] for ch, f in zip(chars, freq)]]
    heapq.heapify(heap)

    while len(heap) > 1:
        f1, left = heapq.heappop(heap)
        f2, right = heapq.heappop(heap)

        for item in left:
            item[1] = "0" + item[1]
        for item in right:
            item[1] = "1" + item[1]

        heapq.heappush(heap, [f1 + f2, left + right])

    return sorted(heap[0][1], key=lambda x: x[0])

chars = input("Enter characters: ").split()
freq = list(map(int, input("Enter frequencies: ").split()))

for ch, code in huffman_codes(chars, freq):
    print(f"{ch}: {code}")
```

Execution / Output

```
Enter characters: A B C D
Enter frequencies: 10 20 30 40
A: 110
B: 111
C: 10
D: 0
```

Result: The program was implemented and the expected result was obtained.

Experiment 13: Huffman Coding – Sensor Data Compression

Objective: To compress IoT sensor symbols using minimum-length prefix codes.

Problem Statement: To compress IoT sensor symbols using minimum-length prefix codes.

Sample Input: Characters = x y z w; Frequencies = [7,8,14,25]

Sample Output: w: 0; z: 10; x: 110; y: 111

Time Complexity: $O(n \log n)$ time and $O(n)$ space.

Explanation: Huffman coding repeatedly combines the two least-frequent nodes to build a prefix-code tree. More frequent symbols receive shorter codes.

Python Program

```
import heapq

def huffman_codes(chars, freq):
    heap = [[f, [[ch, ""]] for ch, f in zip(chars, freq)]
    heapq.heapify(heap)

    while len(heap) > 1:
        f1, left = heapq.heappop(heap)
        f2, right = heapq.heappop(heap)

        for item in left:
            item[1] = "0" + item[1]
        for item in right:
            item[1] = "1" + item[1]

        heapq.heappush(heap, [f1 + f2, left + right])

    return sorted(heap[0][1], key=lambda x: x[0])

chars = input("Enter characters: ").split()
freq = list(map(int, input("Enter frequencies: ").split()))

for ch, code in huffman_codes(chars, freq):
    print(f"{ch}: {code}")
```

Execution / Output

```
Enter characters: x y z w
Enter frequencies: 7 8 14 25
w: 0
z: 10
x: 110
y: 111
```

Result: The program was implemented and the expected result was obtained.

Experiment 14: Coin Change – Vending Machine

Objective: To return the minimum number of coins from a vending machine for a given amount.

Problem Statement: To return the minimum number of coins from a vending machine for a given amount.

Sample Input: Coins = [1,2,5], Amount = 11

Sample Output: 5 5 1; Minimum coins = 3

Time Complexity: $O(n \log n)$ time after sorting.

Explanation: The greedy approach sorts denominations in descending order and repeatedly selects the largest denomination that does not exceed the remaining amount.

Python Program

```
def coin_change(coins, amount):
    coins = sorted(coins, reverse=True)
    result = []

    for coin in coins:
        count = amount // coin
        result.extend([coin] * count)
        amount %= coin

    return result

coins = list(map(int, input("Enter coins: ").split()))
amount = int(input("Enter amount: "))

result = coin_change(coins, amount)
print("Coins used:", result)
print("Minimum coins =", len(result))
```

Execution / Output

```
Enter coins: 1 2 5
Enter amount: 11
Coins used: [5, 5, 1]
Minimum coins = 3
```

Result: The program was implemented and the expected result was obtained.

Experiment 15: Fractional Knapsack – Warehouse Packing

Objective: To maximize warehouse value by selecting whole or fractional portions of items within a fixed capacity.

Problem Statement: To maximize warehouse value by selecting whole or fractional portions of items within a fixed capacity.

Sample Input: Weights = [2,3,5,7], Profits = [10,5,15,7], Capacity = 8

Sample Output: Maximum profit = 30

Time Complexity: $O(n \log n)$ time and $O(n)$ space.

Explanation: Items are sorted by profit-to-weight ratio. The highest-ratio items are selected first, and a fraction of the next item may be taken when the remaining capacity is smaller than its weight.

Python Program

```
def fractional_knapsack(weights, profits, capacity):
    items = sorted(
        [(p / w, w, p) for w, p in zip(weights, profits)],
        reverse=True
    )

    total = 0

    for ratio, weight, profit in items:
        take = min(weight, capacity)
        total += ratio * take
        capacity -= take

        if capacity == 0:
            break

    return total

weights = list(map(int, input("Enter weights: ").split()))
profits = list(map(int, input("Enter profits: ").split()))
capacity = int(input("Enter capacity: "))

print("Maximum profit =", fractional_knapsack(weights, profits, capacity))
```

Execution / Output

```
Enter weights: 2 3 5 7
Enter profits: 10 5 15 7
Enter capacity: 8
Maximum profit = 30.0
```

Result: The program was implemented and the expected result was obtained.